

Reviewer Pack — Minimal Rank of Real Algebraic Curves Bounds AC^0 Circuit Dep...

Entry b78b421ca428 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 22:51:54 UTC

Minimal Rank of Real Algebraic Curves Bounds AC^0 Circuit Depth for Parity

Entry ID: b78b421ca428 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 22:51:54 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): Complexity Theory: AC^0 Circuit Complexity

Statement:

{‘quantitative relation’: ‘ $E[\text{rank}(C)] = \Theta(2^{n/\log^2(n)})$ for all real algebraic curves C that define the Parity function on n inputs.’, ‘invariant requirement’: ‘ $\text{rank}(C)$ is the minimal number of real roots for a curve C that defines Parity, and $E[X]$ denotes the expected value over all such curves.’, ‘counterexample’: ‘For any fixed constant $c > 0$, there exists an instance with $n \leq 40$ such that $\text{rank}(C) < c \cdot n/\log^2(n)$ for some real algebraic curve C defining Parity.’}

Rationale (proposer’s reasoning):

{‘potential structure’: ‘Real algebraic geometry provides a rich framework to analyze the geometric properties of functions. By exploring the minimal rank of real algebraic curves, we may uncover structural properties of computational problems like Parity that are not apparent through traditional complexity measures.’, ‘novelty’: ‘The relationship between real algebraic curves and circuit complexity is relatively unexplored, suggesting a new perspective for understanding the nature of computation.’}

Taxonomy category: $AC0_PARITY$ (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d01eed344210b53d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the mean of minimal ranks $E[\text{rank}(C)]$ over 30 random seeds falls within 20% of $\Theta(2^{n/\log^2(n)})$ with standard deviation $\leq 0.5 \cdot n$ and falsified if any seed’s mean rank deviates by more than 50% from this expected value or a seed produces a curve C with $\text{rank}(C) < c \cdot n/\log^2(n)$ for some constant $c > 0$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=-9, elapsed=73.3s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_polynomial(n):
        coeffs = [random.choice([-1, 1]) for _ in range(n+1)]
        return coeffs

    def evaluate_polynomial(poly, x_values):
        n = len(poly) - 1
        values = []
        for x_val in x_values:
            value = sum(c * (x_val ** i) for i, c in enumerate(reversed(poly)))
            values.append(value)
        return values

    def is_parity_function(values):
        return all(v % 2 == int(x % 2) for v, x in zip(values, range(len(values))))

    def compute_rank(poly, n):
        # Simple heuristic to estimate rank (not rigorous)
        return len(set(evaluate_polynomial(poly, range(2**n))))

    n = random.randint(5, 40)
    poly = generate_polynomial(n)
    x_values = [i for i in range(2**n)]
    values = evaluate_polynomial(poly, x_values)

    if not is_parity_function(values):
        return {
            "metric_name": "minimal_rank",
```

```

        "metric_value": None,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "The polynomial does not define the Parity function."
    }

rank = compute_rank(poly, n)
return {
    "metric_name": "minimal_rank",
    "metric_value": rank,
    "instances_tested": 1,
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    total_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None)
    mean_metric_value = total_metric_value / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))

    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any("counterexample" in r and r["counterexample"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if "counterexample" in r and r["counterexample"])
        print(f"RESULT: FALSIFIED counterexample={first_failing_seed} \"The polynomial does not define the Parity function.\"")
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

(empty)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means we cannot verify if the mean of minimal ranks $E[\text{rank}(C)]$ over 30 random seeds falls within the specified conditions. | next: Re-run the test to ensure it completes successfully and provides the necessary data for analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12074
2	preregistration	ollama_remote	glm4:latest	0	0	6401
3	novelty	ollama_remote	glm4:latest	0	0	4909
4	novelty	ollama_remote	glm4:latest	0	0	5237
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13589
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11911
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8640
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8493
9	judge	ollama_remote	glm4:latest	0	0	75189

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 146444 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/b78b421ca428.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b78b421ca428.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/b78b421ca428.tar.gz` (if generated)